

Analysis: Enclosure

The goal is to place as few stones as possible to enclose at least K intersection points in an $N \times M$ grid. Intuitively, it is wasteful to create more than one enclosure since we can always combine the stones into one larger enclosure that cover same or more intersection points. Thus, we can restrict our search to solutions with one enclosure only.

The figure below shows an enclosure for an $N = 5$ by $M = 5$ grid that covers $K = 19$ intersection points with the minimum number of 11 stones.

```
-***-    # the first row
*XXX*    # intermediate row
*XXX*    # intermediate row
*XX*-    # intermediate row
-***-    # the last row
```

We use '-' to represent empty intersection, 'x' to represent an enclosed intersection point, and '*' to represent a stone placed in the intersection point (which is also enclosed). Observe that:

- The stones in the first row and the last row are filled in consecutive position in the row.
- Each intermediate row (i.e., the row except the first and the last row) has exactly two stones (i.e., the left stone and the right stone).

Each intermediate row always contains two stones. If there exists an intermediate row that contains only one stone, then it may form two enclosures that touches their boundaries at that row. Since we are not interested in searching solutions with more than one enclosure, we can restrict each intermediate row to always contains exactly two stones.

What about having more than two stones in an intermediate row? This is wasteful and we can avoid that too. To see this, let's enumerate all possible ways to move the left stone boundary for the next row and at the same time it explains why each intermediate row always contains two stone in the final configuration. The following figure shows all three possible ways to move the left boundary stone:

```
previous row:  --*XXX      --*XXX      --*XXX
next row:      -*XXXX      --*XXX      ---*XX
                (expand)    (unchanged)  (shrink)
```

Notice that we can only expand the boundary by one otherwise it will create a gap and it will no longer form a **closed** enclosure. Could we place stones in between to close the gap? Such as (shown in bold):

```
previous row:  --*XXX      -**XXX
next row:      **XXXX      *XXXXX
                  (push up)
```

Yes, we can, but it is wasteful because we can always "push up" the placed stone to get one more enclosed intersection point as shown in the right figure. Moreover, we can do another push up on the previous row to the previous-previous row and so on until it is pushed up to the top row (each push up gains one more enclosed intersection point). Thus, in the end, after all the push ups, each intermediate row will contain exactly two stones.

To make the search simple, we do not want any push up to happen. In the search, we fix the number of stones at the top row, and generate the next row and we do not want the next row to alter the previous row ("push up" alters the previous row). This explains why we do not want to expand the left boundary more than one position (i.e., it is simpler just to change the number of stones in the top row and do a separate search).

For the shrink case, observe that we only need to shrink at most one position because shrinking more than one is wasteful since we will need more stones to fill the gaps (to maintain enclosure). See the following examples:

```
previous row:  --*XXX      --*XX
next row:      ---*X       ----*X
              (case 1)    (case 2)
```

In the shrinking case 1, the bolded stone '*' (the rightmost stone) is not needed since it is already enclosed. The shrinking case 2 is wasteful since the bolded stone '*' (the middle stone) can be pushed down to enclose one more intersection point (which will still be wasteful because it will then become case 1).

Looking at all the possible ways to move the left boundary, we can conclude that it is sufficient to place exactly two stones in each intermediate row.

For the last row, we close the enclosure by connecting the left and right stone boundary of the previous row by placing the stones consecutively. For example:

```
previous row:  --*XXXXX*--
last row:      ---*****---
```

Note that we only close the enclosure if we are sure that the intersection points enclosed by all the previous rows and the last row is at least **K**.

In summary, our search algorithm goes as follows:

- First place a number of stones consecutively from left to right on the top row
- Place two stones for the following (intermediate) rows by expanding / shrinking / unchanged the left and right boundary with respect to the previous row
- Finally if we have enclosed enough intersection points, close the enclosure by placing stones consecutively at the last row.

We will discuss two common ways to solve this problem. The first is a dynamic programming (DP) solution and the second is a greedy solution.

Dynamic Programming (DP)

Since we brute-force the first row, we only need to perform DP for the intermediate rows and the last row. For each row, we need to know:

- The remaining rows left.
- The left stone boundary position.
- The right stone boundary position.
- The remaining intersection points to enclose.

To minimize the left / right stone boundary position, we transpose the grid (if necessary) so that we ended up with the grid with smaller columns than its rows, without affecting optimality. With this transformation, now the left / right boundary position is at most $\sqrt{N * M}$. This leads to $O(N * \sqrt{N * M} * \sqrt{N * M} * N * M)$ solution. While the amortized cost can be less, it still seems large and has potential to run more than the time limit. Can we do better?

It turns out that the exact position of the left and right stone boundary does not really matter as long as the distance between them is not larger than the column size. This also applies for the stones at the top row. What matters is the number of stones placed consecutively in the top row. Where the stones are exactly placed does not matter as long as the number of stones is at most the column size.

With this intuition, we can make the DP state smaller. Instead of maintaining the left and right stone boundary position, we can just maintain the distance between the two stones instead. The three possibilities of moving the stone boundaries in the next row (expand, shrink, unchanged) now translate to five possibilities of adding the stone distance, by -2, -1, 0, 1, or 2 as shown in the following examples.

```
prev row:  -*XXX*-  -*XXX*-  -*XXX*-  -*XXX*-  -*XXX*-
next row:  --*X*--  --*XX*-  -*XXX*-  -*XXXX*  *XXXXX*
           (-2)    (-1)    (0)      (1)      (2)
```

Note that for (-1) and (1) there is another possibility for the next row, but both have the same distance between the left and right stone boundaries.

By switching the left and right boundary position to the distance between the left and right stone, we can reduce the DP states to three states (the remaining rows left, stone distance of the previous row, the remaining intersection to enclose). This reduces the complexity to $O(N * \sqrt{N * M} * N * M)$. The amortized cost is less than 32 million operations per test case which is fast enough to answer 100 test cases. Below is a sample implementation in Python 3:

```
from functools import lru_cache
import sys

@lru_cache(maxsize = None) # Memoization.
def rec(rem_rows, prev_dist, rem_points, M):
    if rem_points <= 0: # If the remaining area is non positive,
        return 0 # then no stone is needed.

    ret = 1000000 # Infinity.
    if rem_rows <= 0: # No more row but rem_points is still > 0.
        return ret # Return infinity.

    if M == 1: # Special case where each row only has one stone.
        return rem_points # rem_rows >= rem_points is guaranteed.

    min_dist = max(prev_dist - 2, 1)
    max_dist = min(prev_dist + 2, M)
    for next_dist in range(min_dist, max_dist + 1):
        if next_dist >= rem_points:
            # Close the enclosure for the last row.
            ret = min(ret, next_dist)
        elif next_dist > 1:
            # Cover this row using 2 stones.
            next_rem_points = rem_points - next_dist
            ret = min(ret, \
                2 + rec(rem_rows - 1, next_dist, next_rem_points, M))

    return ret

def min_stones(N, M, K):
    if N < M: # If the row size is smaller than the column size
```

```

(N, M) = (M, N) # Transpose the grid

res = 1000000
# Try all possible number of stones for the top row.
for stones in range(1, min(K, M) + 1):
    # The stones needed to cover the top row + the next rows.
    stones = stones + rec(N - 1, stones, K - stones, M)
    res = min(res, stones)

return res

sys.setrecursionlimit(5000)
for tc in range(int(input())):
    print("Case #%d: %d" % (tc+1, \
        min_stones(*map(int, input().split()))))

```

Greedy

Intuitively, the two stones in each intermediate row can be greedily placed as far as possible from each other to maximize the area enclosed without adding any additional stone. Consider the following example:

-----	-----	-----*
-----	-----	---*X*---
--*****--	--*****--	--*XXX*--
--*XXX*--	--*XXXXX*--	--*XXXXX*--
--*XXX*--	*XXXXXXXX*	*XXXXXXXX*
--*XXX*--	--*XXXXX*--	--*XXXXX*--
--*****--	--*****--	--*XXX*--
-----	-----	---*X*---
-----	-----	-----*
(a)	(b)	(c)

The enclosure in Figure (a) can be improved by moving the two stones (in each of the three intermediate rows) as far as possible from each other as shown in Figure (b). Similar reasoning can be made for each column that contain only two stones: move the top and bottom stones as far from each other as possible in that column. The enclosure in Figure (b) can be improved in the same way, resulting the enclosure as shown in Figure (c).

Knowing that the optimal shape resembles a diamond, the greedy approach is to try to construct a diamond-shaped enclosure with area at least **K**. However, this is not always feasible if the grid is not large enough. In such case, the diamond-shape may be “truncated” at the top / left / right / bottom sides as shown below for the best enclosure for **N** = 6, **M** = 7, **K** = 27

```

--***--
-*XXX*-
*XXXXX*
-*XXXX*
--*XX*-
---**--

```

Another useful observation is that the empty intersections at the corners always forms a right triangle. This allows us to generate all possible truncated (and perfect) diamond by placing empty triangles at the corners. Notice that the sizes (the length of its side) of the empty triangles at the corners may be different by at most one size.

Fortunately, the large input is small enough that we can brute-force for all possible truncated (and perfect) diamond shapes. First we try all possible grid size, and for each possible grid size, we try to put empty triangles at the corners and compute the enclosure size and the stones needed. We record and return the minimum stones needed to construct the shape with area at least **K**. Below is a sample implementation in Python 3:

```
def empty_triangle(size):
    return size * (size + 1) / 2

def min_stones(N, M, K):
    if N > M:
        (N, M) = (M, N)

    best = K
    for R in range(2, N + 1):
        for C in range(R, M + 1):
            if R * C >= K:
                for i in range(2 * R):
                    cover = R * C
                    cover -= empty_triangle(i // 4)
                    cover -= empty_triangle((i + 1) // 4)
                    cover -= empty_triangle((i + 2) // 4)
                    cover -= empty_triangle((i + 3) // 4)
                    if cover < K:
                        break
                    stones = 2 * (R + C) - 4 - i
                    best = min(best, stones)

    return best
```

The complexity of the above solution is $O(N^2 * M)$. However, if you are well versed in Mathematics, you can further improve the search to $O(\log(K))$ by doing a binary search on the number of stones needed to form the truncated diamond shape and compute the number of enclosed intersection points in $O(1)$ to make the binary search decision.

Analysis: Part Elf

In the first generation (i.e., 40 generations ago) there were only either pure Elf ($1/1$ Elf) or Human ($0/1$ Elf). If we enumerate all possible children for the next 3 generations we have:

1st gen:	0/1								1/1
2nd gen:	0/2			1/2					2/2
3rd gen:	0/4		1/4		2/4		3/4		4/4
4th gen:	0/8	1/8	2/8	3/8	4/8	5/8	6/8	7/8	8/8

It is apparent that at any generation, the denominator is always a power of two. Thus, the answer for the **impossible** case is easy to check: first reduce the given fraction to its lowest terms and check whether the denominator is a power of two. A fraction can be reduced to its lowest terms by dividing both the numerator and denominator by their greatest common divisor.

Now, the remaining question is for a P/Q Elf, what is the minimum number of generations ago that there could have been a $1/1$ Elf?

For a small input, where Q is at most 1000, we can generate the first 10 generations (with 2^{10} possible Elf combinations) to cover all possible small input. When we generate the child from the two parent we record their relationship. Then to answer the question, we simply do a breadth-first search (shortest path in unweighted graph) in the relationship graph from the P/Q Elf and stop whenever we encounter $1/1$ Elf and report the length (shortest path length). This algorithm runs in $O(2^N * 2^{2N})$ to generate the relationship graph of size 1001×1001 which is still feasible for a small input but not for the large input.

For the large input, another insight is needed. Let's do some examples to get the intuition. Suppose Vida is an $3/8$ Elf, what are the possible parents? Let's enumerate them:

- $(0/8 + 6/8) / 2 = 3/8$
- $(1/8 + 5/8) / 2 = 3/8$
- $(2/8 + 4/8) / 2 = 3/8$
- $(3/8 + 3/8) / 2 = 3/8$

The possible parents of $3/8$ are: $0/8, 1/8, 2/8, 3/8, 4/8, 5/8, 6/8$ which form a consecutive (for the lack of a better word) fraction from $0/8$ to $6/8$.

Formally speaking, a P/Q Elf could have a Z/Q parent where Z is ranged from $\max(0, P - (Q-P))$ to $\min(Q, P * 2)$. Notice that the denominator Q does not change.

With this intuition, it is obvious that to get to the pure $1/1$ Elf as fast as possible, we want to greedily generate a parent with numerator as large as possible. In other words, from an P/Q Elf we would like to generate Z/Q parent where Z is maximized. We continue the process for Z/Q , picking the parent with the greatest numerator and so on until we get to a $1/1$ Elf. This greedy algorithm runs in $O(\log(Q))$. Below is a sample implementation in Python 3:

```
from fractions import gcd

def is_power_of_two(x):
    return x & (x - 1) == 0

def min_generation(P, Q):
    g = gcd(P, Q)
```

```
P = P // g
Q = Q // g

if not(is_power_of_two(Q)):
    return "impossible"

gen = 0
while P < Q:
    P = P * 2
    gen += 1
return gen

for tc in range(int(input())):
    print("Case #%d: %s" % (tc+1, \
        min_generation(*map(int, input().split('/')))))
```

Analysis: Reordering Train Cars

This is a straightforward problem, but the main issue to get it correct is to enumerate all invalid cases. The main idea of the solution is to join some strings to create sets of disjoint groups, such that every group contains a set of strings that must be joined together. Then calculate the number of ways to create every group, and the final result will be the factorial of number of disjoint groups multiplied by number of ways to create every group.

One of the main issues (that lead to huge number of incorrect attempts) was to validate that the given set of strings is invalid. So let us start the analysis by specifying valid cases that should be handled before creating the groups.

Another good hint that will help a lot in validation process is to collapse all given strings by removing all duplicate consecutive letters, for example, if given a string "aabbccdddeaa", after collapsing this string it should be "abcdea". So from now, we will assume that all given strings are always collapsed, even after joining two strings "ab" and "bc" this will be collapsed automatically to be "abc" not "abbc".

Validation Process

First, check that for every string there is no letter repeated in two different places in the string. For example, ["aba", "abca", "adeab"] are all invalid strings as 'a' is repeated, while ["ab", "abc", "adbe"] are valid strings.

We start by precomputing some information that will help to enumerate the rest of invalid cases. We need to store the position of every character **c** in the alphabets, i.e. all letters from 'a' to 'z'. Character **c** will be either (i) the first character, (ii) the last character, or (iii) a middle character of any given non-single-character string. Also, we need to count the number of single-character strings for every character **c**. For example, if the given list of collapsed strings is ["abc", "cdef", "a", "a", "gh"], then the precomputed arrays are:

```
begin = {'a': 0, 'c': 1, 'g': 4}    -- (1)
end = {'c': 0, 'f': 1, 'h': 4}    -- (2)
middle = {'b': 0, 'd': 1, 'e': 1} -- (3)
singleChars = {'a': 2}            -- (4)
```

Notes:

1. **begin** means that for any character **c**, it is the first character of string number **begin[c]**.
2. **end** means that for any character **c**, it is the last character of string number **end[c]**.
3. **middle** means that for any character **c**, it is in the middle of string number **middle[c]**. Notice that, "cdef" (string number 1) has two middle characters, while "gh" (string number 4) does not have middle characters.
4. **singleChars** means that for any character **c**, There are **singleChars[c]** single-character strings composed of character **c**. For example, there are 2 single-character strings of character 'a'.

While precomputing the previous arrays, you have identify a valid set of strings by sustaining the following conditions. A character is valid only if it satisfies one of the following conditions:

1. It appears in the middle of one string, and nowhere else.

2. It appears at most once in the beginning of one string, at most once at the end of one string, and any number of times as a single-character string.

For example, all the following cases are invalid:

- ["abc", "ade"]: character 'a' conflicts with second condition.
- ["bca", "dea"]: character 'a' conflicts with second condition.
- ["abc", "dbf"]: character 'b' conflicts with second condition.
- ["abc", "ead"]: character 'a' conflicts with first condition.
- ["abc", "gcf"]: character 'c' conflicts with first condition.
- ["a", "a", "bac"]: character 'a' conflicts with first condition.

By considering the previous conditions, we will be able to identify valid cases in any given set of strings.

Groups creation

Let us define first what it is a group, the group is a set of strings that should be joined to form a valid combined string. So in this section we focus on creating a set of disjoint groups to count the number of ways. For example, ["ab", "bc"] is a group as "ab" and "bc" should be joined together to form "abc", which is a valid string. Also, assume given ["ab", "bc", "de", "xy", "yz"], this set of strings can be divided into 3 disjoint groups, which are ["ab", "bc"], ["de"], and ["xy", "yz"].

First, we need to handle the special case of single-character strings. Every single-character string can be considered initially as a group, with the number of ways to create it being equal to the factorial of the number of such strings, for example, ["a", "a", "a"] can be grouped in 3! (which is 6 different ways). And this group will be represented by only one character which is "a" and not "aaa" (as we described in the beginning, we will collapse these strings too).

An important point to notice while creating groups is that the first string in the group should start with a character that is not used as the last character in any other string. For example, the following 2 strings ["abc", "cde"] can form a valid group that should be represented by string "abcde", we notice that we cannot start forming the group with string "cde" as it starts with character 'c' which is the last character of string "abc" (we can make this check in constant time by using the *end* array). Also, notice that this group can be created in only 1 way.

We now proceed with counting the *number of disjoint groups*. We loop over all non-single-character strings that start with a character not used as the last character of any other string and use this as the first string in the group, then join it to the string that starts with its last character (get the index of this string using *begin* array) and so on till you reach the last string in that group (where you cannot find any other string to join it to). For example:

```
["cde", "mno", "abc", "opq", "xyz"]
```

From these strings we can form 3 disjoint groups as follows:

- 1) "mno" + "opq" = "mnopq"
- 2) "abc" + "cde" = "abcde"
- 3) "xyz"

After counting the number of disjoint groups there is one more step remaining which is to join any single-character string to an disjoint group, if possible. As stated previously, we assumed initially that every single-character string is considered as a disjoint group, so we may assume that total number of disjoint groups equals to number of non-single-character groups and number of single-character groups. But there are some cases that we need to join a single-character strings to a one of the non-single-character strings. We can elaborate that with the following two examples:

- ["ab", "b", "b"]: you must join the single-character string "b" to the end of string "ab" to form the group "ab" in 2 ways, i.e. 2 ways for the 2 single-character strings and only 1 way to join the single-character string to "ab".
- ["a", "a", "bc"]: in this case we consider "a" as an disjoint group and will not be joined to any other groups. So there are 4 ways, i.e. 2 ways for the single-character string, and 2 ways to order "a" and "bc" either "abc" or "bca".